Unit 4
Client Side Scripting language

Prepared By : Yachu Shakya

Introduction to JavaScript

- **JavaScript** was invented by **Brendan Eich** in 1995.
- It was developed for **Netscape 2**, and became the **ECMA-262** standard in 1997.
- JavaScript is the scripting language of the Web.
- JavaScript is used in millions of Web pages to add functionality, validate forms, detect browsers, and much more.
- JavaScript is the most popular scripting language on the Internet, and works in all major browsers, such as Internet Explorer, Mozilla Firefox, Chrome, Opera and many more.

What is JavaScript?

- JavaScript was designed to add interactivity to HTML pages
- JavaScript is a scripting language
- A scripting language is a lightweight programming language
- JavaScript is usually embedded directly into HTML pages
- Everyone can use JavaScript without purchasing a license
- Loosely tightened

Java and JavaScript are two completely different languages in both concept and design!

What can a JavaScript Do ?

- **JavaScript gives HTML designers a programming tool -** HTML authors are normally not programmers, but JavaScript is a scripting language with a very simple syntax! Almost anyone can put small "snippets" of code into their HTML pages
- **JavaScript can put dynamic text into an HTML page** - A JavaScript statement like this: document. write(**“<h1>” + name + “</h1>**") can write a variable text into an HTML page
- **JavaScript can react to events** - A JavaScript can be set to execute when something happens, like when a page has finished loading or when a user clicks on an HTML element
- **JavaScript can read and write HTML elements -** A JavaScript can read and change the content of an HTML element
- **JavaScript can be used to validate data - A** JavaScript can be used to validate form data before it is submitted to a server. This saves the server from extra processing
- **JavaScript can be used to detect the visitor's browser -** A JavaScript can be used to detect the visitor's browser, and - depending on the browser - load another page specifically designed for that browser
- **JavaScript can be used to create cookies -** A JavaScript can be used to store and retrieve information on the visitor's computer

JavaScript Variables

- Variables are "containers" for storing information.
- JavaScript variables are used to hold values or expressions.
- A variable can have a short name, like x, or a more descriptive name, like carname.
- Rules for JavaScript variable names:
  - Variable names are case sensitive (y and Y are two different variables)
  - Variable names must begin with a letter or the underscore character

Note: Because JavaScript is case-sensitive, variable names are case-sensitive.

JavaScript Variables

- var
- let
- const

Declaring (Creating) JavaScript Variables using var

Creating variables in JavaScript is most often referred to as "declaring" variables.

You can declare JavaScript variables with the **var statement**:

```
var x;
var carname;
```

After the declaration shown above, the variables are empty (they have no values yet).

However, you can also assign values to the variables when you declare them:

```
var x=5;
var carname="Scorpio";
```

After the execution of the statements above, the variable **x** will hold the value **5**, and **carname** will hold the value **Scorpio**.

Note: When you assign a text value to a variable, use quotes around the value.

Redeclaring JavaScript Variables

If you redeclare a JavaScript variable, it will not lose its original value.

```
var x=5;
var x;
```

Assigning Values to Undeclared JavaScript Variables

If you assign values to variables that have not yet been declared, the variables will automatically be declared.

These statements:

```
x=5;
carname="Scorpio";
```

have the same effect as:

```
var x=5;
var carname="Scorpio";
```

Example

```
<html>
<body>
<script type="text/javascript">
var firstname;
firstname="Welcome";
document.write(firstname);
document.write("<br />");
firstname="XYZ";
document.write(firstname);
</script>
<p>The script above declares a variable,
assigns a value to it, displays the value, change the value,
and displays the value again.</p>
</body>
</html>
```

Welcome
XYZ

The script above declares a variable, assigns a value to it, displays the value, change the value, and displays the value again.

JavaScript Let

- The let keyword was introduced in ES6 (2015)
- Variables defined with let cannot be Redeclared.
- Variables defined with let must be Declared before use
- Variables defined with let have Block Scope

With let you can **not** do this:

```
let x = "John Doe";

let x = 0;
```

With var you can:

```
var x = "John Doe";

var x = 0;
```

JavaScript Const

- The const keyword was introduced in ES6 (2015)
- Variables defined with const cannot be Redeclared
- Variables defined with const cannot be Reassigned
- Variables defined with const have Block Scope

Data Types

- **Numbers -** are values that can be processed and calculated. You don't enclose them in quotation marks. The numbers can be either positive or negative.
- **Strings -** are a series of letters and numbers enclosed in quotation marks. JavaScript uses the string literally; it doesn't process it. You'll use strings for text you want displayed or values you want passed along.
- **Boolean (true/false) -** lets you evaluate whether a condition meets or does not meet specified criteria.
- **Null -** is an empty value. null is not the same as 0 -- 0 is a real, calculable number, whereas null is the absence of any value.
- **Undefined**

Example

```
<!DOCTYPE html>
<html>
<body>
<h2>Datatypes</h2>
<script>
let isActive = true; // boolean
let name = "John"; // string
let age = 30; // number
let address = null; // null

document.write("Name: " + name);
document.write("Age: " + age);
document.write("Address: " + address);
document.write("Is Active: " + isActive);
</script>
</body>
</html>
```

Datatypes

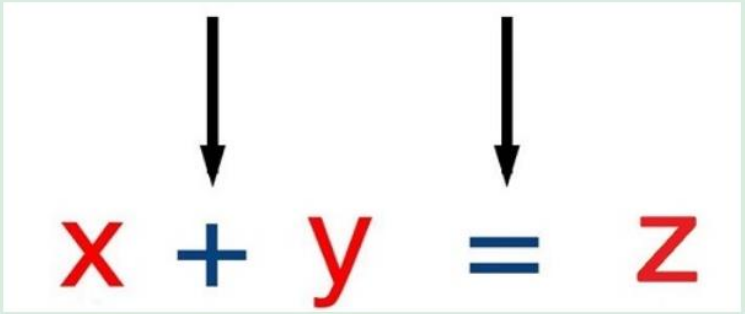
Name: JohnAge: 30Address: nullIs Active: true

JavaScript Operators

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation (ES2016)
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

The **Addition Operator** + adds numbers:

The **Assignment Operator** = assigns a value to a variable.



Javascript Statements

- A computer program is a list of instructions to be executed by a computer
- In programming language, these programming instructions are called statements.
- JavaScript program is a list of programming statements
- JavaScript statements are composed of:
- Values, Operators, Expressions, Keywords and Comments

JavaScript Statements

- JavaScript statements are terminated by “;” semicolon.
- JavaScript ignores white spaces.
- If a JavaScript statement does not fit on one line, the best place to break it is after an operator

JavaScript Expression

- JavaScript expression is composed up of values, variables and operators .
- Eg. A*5
- B=A+B;

JavaScript Keywords

- Keywords are reserved words in JavaScript Library

Keyword	Description
var	Declares a variable
let	Declares a block variable
const	Declares a block constant
if	Marks a block of statements to be executed on a condition
switch	Marks a block of statements to be executed in different cases
for	Marks a block of statements to be executed in a loop
function	Declares a function

JavaScript Blocks

- In JavaScript, statements can be grouped together in blocks inside curly brackets {……} .
- Motive is to define and execute statements together.
- Eg. Using function.

Arithmetic Operators

Arithmetic operators are used to perform arithmetic between variables and/or values.

Given that **y=5**, the table below explains the arithmetic operators:

Operator	Description	Example	Result
+	Addition	x=y+2	x=7
-	Subtraction	x=y-2	x=3
*	Multiplication	x=y*2	x=10
/	Division	x=y/2	x=2.5
%	Modulus (division remainder)	x=y%2	x=1
++	Increment	x=++y	x=6
--	Decrement	x=--y	x=4

Assignment Operator

Assignment operators are used to assign values to JavaScript variables.

Given that **x=10** and **y=5**, the table below explains the assignment operators:

Operator	Example	Same As	Result
=	x=y		x=5
+=	x+=y	x=x+y	x=15
-=	x-=y	x=x-y	x=5
*=	x*=y	x=x*y	x=50
/=	x/=y	x=x/y	x=2
%=	x%=y	x=x%y	x=0

Comparison Operator

Comparison operators are used in logical statements to determine equality or difference between variables or values.

Given that **x=5**, the table below explains the comparison operators:

Operator	Description	Example
==	is equal to	x==8 is false
===	is exactly equal to (value and type)	x===5 is true x==="5" is false
!=	is not equal	x!=8 is true
>	is greater than	x>8 is false
<	is less than	x<8 is true
>=	is greater than or equal to	x>=8 is false
<=	is less than or equal to	x<=8 is true

Logical Operators

Logical operators are used to determine the logic between variables or values.

Given that **x=6 and y=3**, the table below explains the logical operators:

Operator	Description	Example
&&	and	(x < 10 && y > 1) is true
||	or	(x==5 || y==5) is false
!	not	!(x==y) is true

Conditional Statements

Very often when you write code, you want to perform different actions for different decisions. You can use conditional statements in your code to do this.

In JavaScript we have the following conditional statements:

- **if statement** - use this statement if you want to execute some code only if a specified condition is true
- **if...else statement** - use this statement if you want to execute some code if the condition is true and another code if the condition is false
- **if...else if....else statement** - use this statement if you want to select one of many blocks of code to be executed
- **switch statement** - use this statement if you want to select one of many blocks of code to be executed

JavaScript Controlling(Looping) Statements

Loops in JavaScript are used to execute the same block of code a specified number of times or while a specified condition is true.

JavaScript Loops

Very often when you write code, you want the same block of code to run over and over again in a row. Instead of adding several almost equal lines in a script we can use loops to perform a task like this.

In JavaScript there are two different kind of loops:

- **for** - loops through a block of code a specified number of times
- **while** - loops through a block of code while a specified condition is true

JavaScript function

- In JavaScript, a function is a block of reusable code that performs a specific task. It is a fundamental building block of JavaScript programming and allows you to encapsulate a set of instructions that can be executed multiple times throughout your code.
- Functions in JavaScript can be defined using the function keyword followed by a name, a list of parameters (optional), and a block of code enclosed in curly braces. Here's a basic syntax for creating a JavaScript function:

```
function functionName(parameter1, parameter2, ...) {
  // Code to be executed
  // Optionally, return a value
}
```

JavaScript function

Let's break down the components of a function:

- function: The keyword used to define a function.
- functionName: The name of the function, which should be unique within the scope of your code.
- parameters: Optional inputs that the function can accept. They are separated by commas and act as placeholders for values that will be passed into the function when it is called.
- code: The block of code that gets executed when the function is called. It can perform operations, manipulate data, or perform any other desired actions.
- return: An optional keyword that allows the function to send a value back to the caller. If omitted, the function returns undefined.

document.write() function

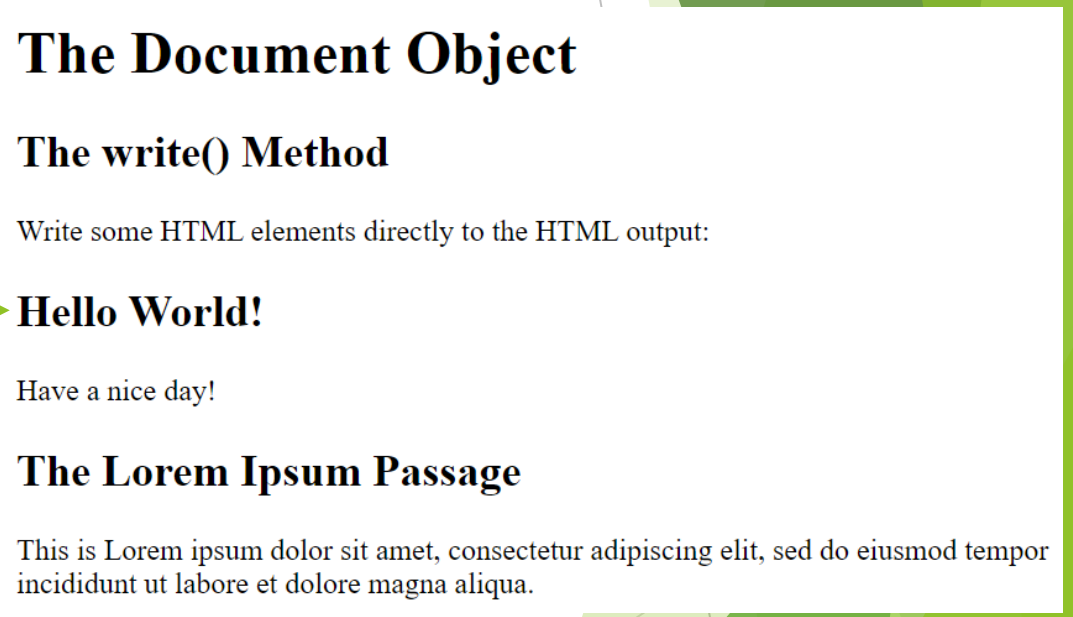
- The document.write() function in JavaScript is used to dynamically generate content and write it directly to the web page. It can be used to display text, HTML, or JavaScript code.
- Here's the basic syntax of the document.write() function:

document.write(content);

The content parameter represents the content you want to write to the document. It can be a string of text, HTML tags, or even JavaScript code.

Example

```
<!DOCTYPE html>
<html>
<body>

<h1>The Document Object</h1>
<h2>The write() Method</h2>

<p>Write some HTML elements directly to the HTML output:</p>

<script>
document.write("<h2>Hello World!</h2><p>Have a nice day!</p>");
</script>

<h2>The Lorem Ipsum Passage</h2>
<p>
This is Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do
eiusmod tempor incididunt ut labore et dolore magna aliqua.

</body>
</html>
```



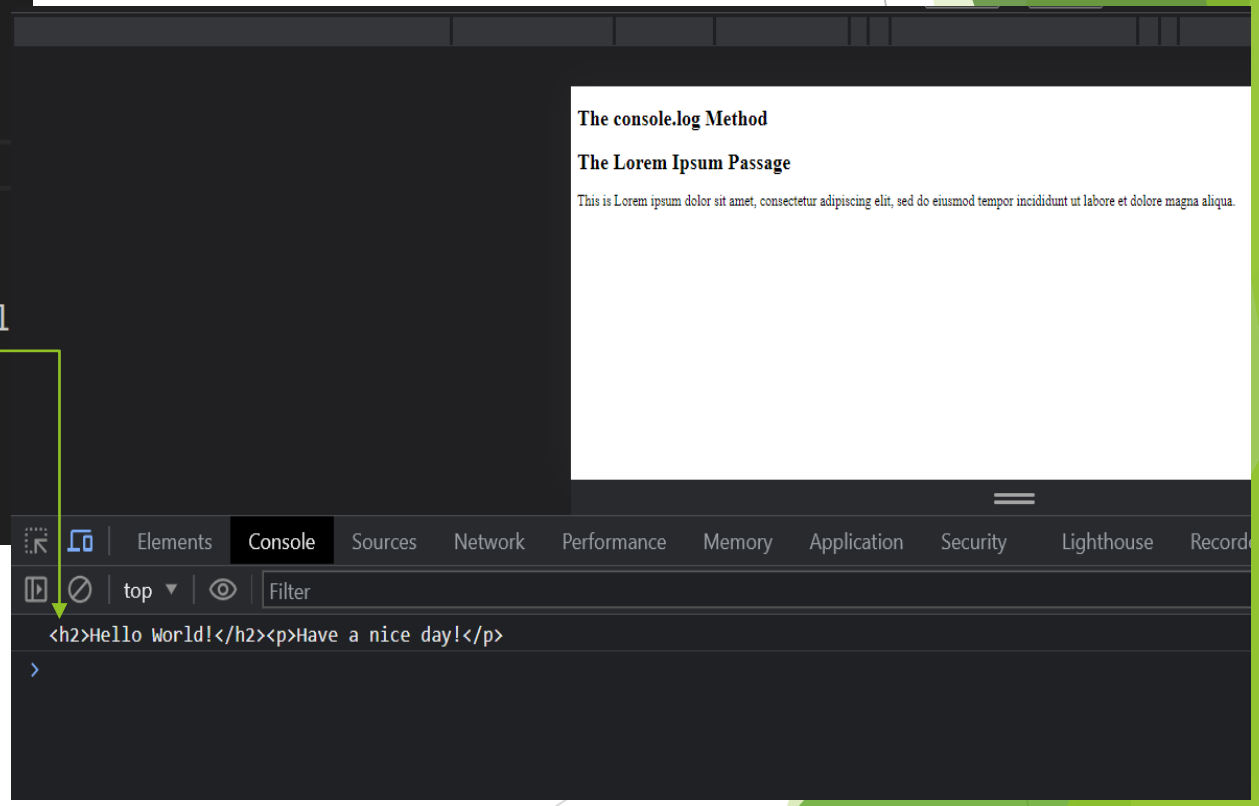
console.log() function

- The console.log() function is a method provided by most web browsers' developer tools and JavaScript runtime environments. It allows you to print messages or values to the console for debugging and logging purposes. The output appears in the browser's developer console, which can be accessed by opening the browser's developer tools.
- Here's the basic syntax of console.log():

console.log(*message*)

Example

```
<!DOCTYPE html>
<html>
<body>
<h2>The console.log Method</h2>
<script>
console.log("<h2>Hello World!</h2><p>Have a nice day!</p>");
</script>

<h2>The Lorem Ipsum Passage</h2>
<p>
This is Lorem ipsum dolor sit amet, consectetur adipiscing el

</body>
</html>
```



getElementById function

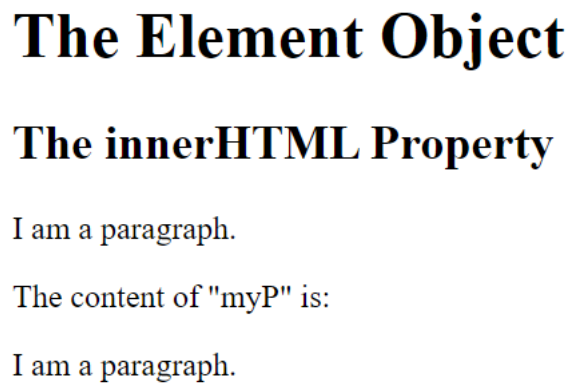
- In JavaScript, the getElementById function is used to retrieve an HTML element from the document based on its unique identifier, known as the "id" attribute. Here's an example of how you can use getElementById:

```
<!DOCTYPE html>
<html>
<body>

<h1 id="demo">The Document Object</h1>
<h2>The getElementById() Method</h2>

<script>
let myElement = document.getElementById("demo");
myElement.style.color = "red";
</script>

</body>
</html>
```

innerHTML property

- The innerHTML property in JavaScript allows you to get or set the HTML content of an element. It represents the markup contained within the element, including any child elements, text, or HTML tags.
- To get the HTML content of an element using innerHTML, you can access the property as shown in the following example:

```
<!DOCTYPE html>
<html>
<body>
<h1>The Element Object</h1>
<h2>The innerHTML Property</h2>

<p id="myP">I am a paragraph.</p>

<p>The content of "myP" is:</p>
<p id="demo"></p>

<script>
let html = document.getElementById("myP").innerHTML;
document.getElementById("demo").innerHTML = html;
</script>

</body>
</html>
```



parseFloat

- parseFloat is a built-in JavaScript function that parses a string and returns a floating-point number. It is commonly used to convert a string containing a numeric value into its corresponding floating-point representation.
- The syntax for parseFloat is as follows:

  parseFloat(string)

- Here, string is the input string that you want to parse into a floating-point number. The function will attempt to extract a numeric value from the string.
- The parseFloat function works by examining the characters in the input string from left to right until it encounters a character that is not a valid part of a number. It then returns the numeric value extracted up to that point.

parseFloat

- Here's an example that demonstrates the usage of parseFloat:

```
var numberString = '3.14';
var floatNumber = parseFloat(numberString);
console.log(floatNumber); // Output: 3.14
console.log(typeof floatNumber); // Output: "number“
```

- In this example, the string '3.14' is successfully parsed by parseFloat, resulting in the floating-point number 3.14. The typeof operator confirms that the returned value is of type "number".
- It's important to note that parseFloat may return NaN (Not a Number) if the string cannot be successfully parsed into a valid number.

Class and Object

- In JavaScript, class and object are fundamental concepts used in object-oriented programming (OOP).
- A class is a blueprint or a template for creating objects. It defines the properties and behaviors that objects of that class will have. You can think of a class as a cookie cutter, while objects are the cookies that are created using that cutter.
- Here's an example of defining a class in JavaScript:

Creating object using object literals

- **We create object using object literal method**
- Syntax:
- Object={ property1:value1,property2:value2;……property:valueN}
- Here, property and value is separated by :(colon) and two property is separated by comma.
- Accessing property
  - Objectname.propertyname;

```
<html>
<head>
    <title>title</title>
    <script>
        //object literal
        let person={
                first_name:"Abisekh",
                last_name:"Chettri",
                getname:function(){
                    alert(person.first_name+' '+person.last_name);
                }
            } // end of object defination
            person.getname();
    </script>
</head>
</html>
```

Creating objects with function constructors

Constructors in JavaScript, like in most other OOP languages, provides a template for creation of objects.

In other words, it defines a set of properties and methods that would be common to all objects initialized using the constructor.

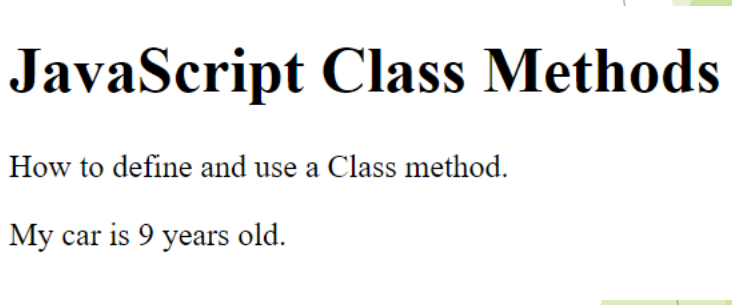
For this we need to create function with argument, Each argument value can be assigned in the current object by using the keyword.

This keyword always refer to current object.

```
<html>
<head>
    <title>title</title>
    <script>
        function Student(name,address,birthyear){
            this.name=name;
            this.address=address;
            this.birthyear=birthyear;
        }
        let student1=new Student('Rajesh','Kathmandu',2000);
        //object student1 created
        let student2=new Student('Ramit','Pokhara',1998);
        document.write(student1.name);
        document.write(student2.name);
    </script>
</head>
</html>
```

Creating objects with class

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Class Methods</h1>
<p>How to define and use a Class method.</p>
<p id="demo"></p>
<script>
class Car {
  constructor(name, year) {
    this.name = name;
    this.year = year;
  }
  age() {
    const date = new Date();
    return date.getFullYear() - this.year;
  }
}
const myCar = new Car("Ford", 2014);
document.getElementById("demo").innerHTML =
"My car is " + myCar.age() + " years old.";
</script>

</body>
</html>
```



Built-in objects in JavaScript

- JavaScript has several built-in objects that provide a wide range of functionality. These objects are part of the JavaScript language itself and are available for use without requiring additional code or libraries. Here are some commonly used built-in objects in JavaScript:
- **Array:** The Array object is used to store and manipulate collections of elements. It provides methods for adding, removing, and accessing elements in an array.
- **String:** The String object represents a sequence of characters and provides methods for manipulating and working with strings.
- **Number:** The Number object represents numeric values and provides methods for working with numbers, such as mathematical operations.
- **Boolean:** The Boolean object represents a Boolean value (true or false) and provides methods for working with logical operations.

Built-in objects in JavaScript

- **Date:** The Date object is used for working with dates and times. It provides methods for creating, manipulating, and formatting dates.

- **Math:** The Math object provides mathematical functions and constants. It includes methods for tasks like rounding numbers, generating random numbers, and performing complex calculations.

- These are just a few examples of the built-in objects available in JavaScript. Each object has its own set of properties and methods that provide specific functionality. You can refer to the JavaScript documentation for more details on these objects and their usage.

JavaScript Array

- In JavaScript, an array is a built-in data structure that allows you to store multiple values in a single variable. Arrays can contain elements of any type, including numbers, strings, objects, and even other arrays. Here's how you can work with arrays in JavaScript:

  Creating an array:

  ```
  const myArray = []; // An empty array

  const numbers = [1, 2, 3, 4, 5]; // An array of numbers

  const fruits = ["apple", "banana", "orange"]; // An array of strings

  const mixedArray = [1, "two", { key: "value" }]; // An array with mixed types
  ```

Accessing elements in an array:

```
const numbers = [1, 2, 3, 4, 5];
console.log(numbers[0]); // Output: 1 (accessing the first element)
console.log(numbers[2]); // Output: 3 (accessing the third element)
console.log(numbers.length); // Output: 5 (getting the length of the array)
```

Modifying elements in an array:

```
let fruits = ["apple", "banana", "orange"];
fruits[1] = "grape"; // Modifying the second element
console.log(fruits); // Output: ["apple", "grape", "orange"]

fruits.push("kiwi"); // Adding an element to the end of the array
console.log(fruits); // Output: ["apple", "grape", "orange", "kiwi"]

fruits.pop(); // Removing the last element from the array
console.log(fruits); // Output: ["apple", "grape", "orange"]
```

Array methods and operations:

JavaScript provides various built-in methods and operations for working with arrays. Some commonly used ones include:

- **push**(): Adds one or more elements to the end of an array.
- **pop**(): Removes the last element from an array and returns it.
- **shift**(): Removes the first element from an array and returns it.
- **unshift**(): Adds one or more elements to the beginning of an array.
- **concat**(): Combines two or more arrays.
- **slice**(): Returns a new array with a portion of the original array.
- **splice**(): Changes the contents of an array by removing, replacing, or adding elements.
- **indexOf**(): Returns the first index at which a given element can be found in the array.
- **includes**(): Checks if an array contains a certain element.
- **join**(): Joins all elements of an array into a string.
- **sort**(): Sorts the elements of an array in place.
- **reverse**(): Reverses the order of elements in an array.

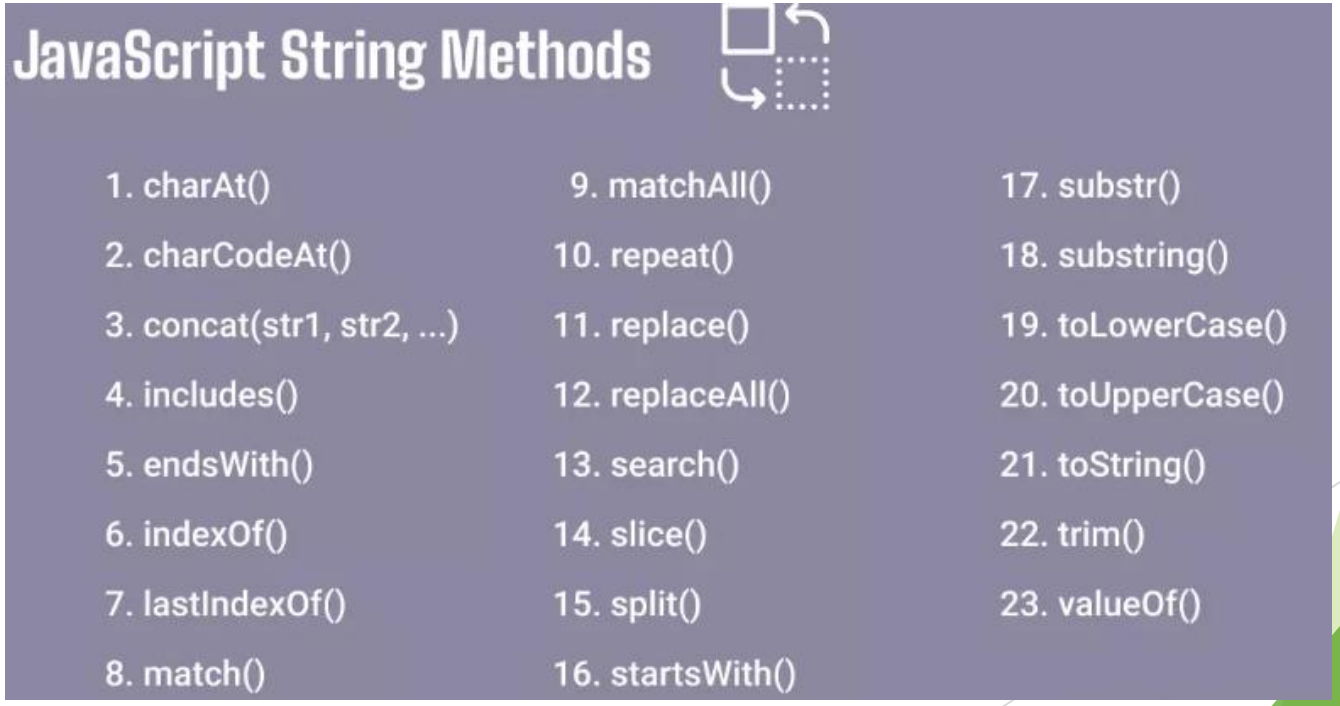
JavaScript String

- In JavaScript, a string is a primitive data type used to represent textual data. It is a sequence of characters, such as letters, numbers, symbols, and whitespace, enclosed within single quotes ('), double quotes ("") or backticks (``). Here are some examples of string literals:

```
let message = "Hello, world!"; // using double quotes

let name = 'John Doe'; // using single quotes

let template = `My name is ${name}`; // using backticks for template literals
```

JavaScript String Methods

- JavaScript provides a wide range of built-in string methods that allow you to manipulate and perform operations on strings. Here are some commonly used string methods:



String Length

- The length property is not a method but a property of a string object in JavaScript. It returns the number of characters in a string. Here's an example:

```
let message = "Hello, world!";

let messageLength = message.length;

console.log(messageLength); // Output: 13
```

- In the example above, message is a string with the value "Hello, world!". The length property is accessed using dot notation (message.length), which returns the length of the string, i.e., the number of characters in the string.
- The resulting value is then stored in the messageLength variable and printed to the console, which will output 13 because the string "Hello, world!" contains 13 characters, including spaces and punctuation.

toUpperCase() and toLowerCase()

- The toUpperCase() method is used to convert a string to uppercase. It returns a new string with all alphabetic characters in the original string converted to uppercase. Non-alphabetic characters remain unchanged.
- The toLowerCase() method is used to convert a string to lowercase. It returns a new string with all alphabetic characters in the original string converted to lowercase. Non-alphabetic characters remain unchanged.

```
let str = "Hello, World!";

let uppercaseStr = str.toUpperCase();

console.log(uppercaseStr); // Output: "HELLO, WORLD!"

let lowercaseStr = str.toLowerCase();

console.log(lowercaseStr); // Output: "hello, world!"
```

concat()

- In JavaScript, the concat() function can also be used to concatenate strings. It allows you to concatenate two or more strings and returns a new string containing the concatenated values.
- Here's the syntax for using concat() to concatenate strings:

```
let firstName = "John";

let lastName = "Doe";

let fullName = firstName.concat(" ", lastName);

console.log(fullName); // Output: John Doe
```

slice()

- In JavaScript, the slice() function is used to extract a portion of a string or an array and return a new string or array containing the extracted elements. The slice() function does not modify the original string or array; instead, it creates a new string or array containing the specified portion.
- Syntax for using slice() with strings:

string.slice(startIndex, endIndex);

Example

```
var sentence = "The quickl brown fox jumps over the lazy dog.";

var extracted = sentence.slice(4, 9);
document.write(extracted); //output:quick
```

replace()

- In JavaScript, the replace() function is used to replace occurrences of a specified value or pattern within a string with a new value. The replace() function returns a new string with the replacements made.
- Here's the syntax for using replace():

string.replace(searchValue, newValue);

Example:

```
var sentence = "The quick brown fox jumps over the lazy dog.";
var newSentence = sentence.replace("quick", "slow");
console.log(newSentence);
```

Output: The **slow** brown fox jumps over the lazy dog.

Visit :https://www.w3schools.com/js/js_string_methods.as

Boolean objects

JavaScript provides the Boolean and Number as object wrappers for Boolean true/false values and numbers, respectively. These wrappers define methods and properties useful in manipulating Boolean values and numbers.

When JavaScript program requires a Boolean value, JavaScript automatically creates a Boolean object to store the value. JavaScript programmers can create Boolean objects explicitly with the statement.

var b=new Boolean(Boolean value);

var b=new Boolean(true);

Boolean object methods

1. **toString():** This method is used to return a string either “true” or “false” depending upon the value of specifies Boolean object.

   var b=true;

   var a=b.toString() // returns Boolean value as string “true” to variable a

2. **tovalueOf()**: This method is used to return a Boolean value either “true” or “false” depending upon the value of specifies Boolean object.

   b.toString() // returns value a Boolean value “true” or “false”

Number objects

- JavaScript automatically creates Number objects to store numeric values in a script. You can create a Number object with the statement
- var n=new Number(value);
- The constructor argument value is the number to store in the object. Although you can explicitly create Number objects, normally the JavaScript interpreter creates them as needed.

Property	Description
EPSILON	The difference between 1 and the smallest number > 1.
MAX_VALUE	The largest number possible in JavaScript
MIN_VALUE	The smallest number possible in JavaScript
MAX_SAFE_INTEGER	The maximum safe integer (2^{53} - 1)
MIN_SAFE_INTEGER	The minimum safe integer -(2^{53} - 1)
POSITIVE_INFINITY	Infinity (returned on overflow)
NEGATIVE_INFINITY	Negative infinity (returned on overflow)
NaN	A "Not-a-Number" value

Number object methods

1. **toExponential()** :this method returns a string representation of a number with rounded and written in exponential form. The uniqueness of each method is in the arguments it accepts. The toExponential() method has an optional argument that can be used to set how many significant digits to use.

   Example:

   ```
   let n=12.6798;
   alert(n.toExponential(2)); // return string 12.68e+0
   alert(n.toExponential(3)); // return string 12.679e+0
   ```

2. **toFixed()** : this method determines the post decimal precision based on the argument passed.

   ```
   let n=12.6798;
   n.toFixed(0); // return 13
   n.toFixed(2); // return 12.68
   ```

Number object methods

3. **toPrecision**(): The toPrecision method determines the significant digits to display based on the argument passed.

```
let n=12.6798;
n.toPrecision();  // return 12.9798
n.toPrecision(3);  // return 12.9
```

4. **toString**() : This method returns a string representation of a number(in this case), but in other objects, it returns a string representation of that object type

```
let a =298;
a.toString(); // return string 298 from variable a
```

5. **valueOf**() : this method returns the primitive value of object type that calls it in this case, the Number object.

```
let a=298;
a.valueOf(); //returns 298 from a
(150+148).valueOf(); // returns 298 from (150+148)
```

Number object methods

6. **ToLocateString():** returns a string value version of the current number in a format that may vary according to a browser's locate settings.

Math object methods

- JavaScript math object is a top level, a predefined object for mathematical constants and functions.
- This object cannot be created by the user. It is a predefined object.
- Mathematical properties and functions can be calculated by math.property or Math.method

```
Math.E          // returns Euler's number
Math.PI         // returns PI
Math.SQRT2      // returns the square root of 2
Math.SQRT1_2    // returns the square root of 1/2
Math.LN2        // returns the natural logarithm of 2
Math.LN10       // returns the natural logarithm of 10
Math.LOG2E      // returns base 2 logarithm of E
Math.LOG10E     // returns base 10 logarithm of E
```

Math object methods

Number to Integer

There are 4 common methods to round a number to an integer:

Math.round(x)	Returns x rounded to its nearest integer
Math.ceil(x)	Returns x rounded up to its nearest integer
Math.floor(x)	Returns x rounded down to its nearest intege
Math.trunc(x)	Returns the integer part of x (new in ES6)

JavaScript Math Methods

Method	Description
abs(x)	Returns the absolute value of x
acos(x)	Returns the arccosine of x, in radians
acosh(x)	Returns the hyperbolic arccosine of x
asin(x)	Returns the arcsine of x, in radians
asinh(x)	Returns the hyperbolic arcsine of x
atan(x)	Returns the arctangent of x as a numeric value between -PI/2 and PI/2 radians
atan2(y, x)	Returns the arctangent of the quotient of its arguments
atanh(x)	Returns the hyperbolic arctangent of x
cbrt(x)	Returns the cubic root of x
ceil(x)	Returns x, rounded upwards to the nearest integer
cos(x)	Returns the cosine of x (x is in radians)
cosh(x)	Returns the hyperbolic cosine of x
exp(x)	Returns the value of E^x

floor(x)	Returns x, rounded downwards to the nearest integer
log(x)	Returns the natural logarithm (base E) of x
max(x, y, z, ..., n)	Returns the number with the highest value
min(x, y, z, ..., n)	Returns the number with the lowest value
pow(x, y)	Returns the value of x to the power of y
random()	Returns a random number between 0 and 1
round(x)	Rounds x to the nearest integer
sign(x)	Returns if x is negative, null or positive (-1, 0, 1)
sin(x)	Returns the sine of x (x is in radians)
sinh(x)	Returns the hyperbolic sine of x
sqrt(x)	Returns the square root of x
tan(x)	Returns the tangent of an angle
tanh(x)	Returns the hyperbolic tangent of a number
trunc(x)	Returns the integer part of a number (x)

Date object

- It is used to work with dates and times. The date object is created by using new keyword. i.e. new Date().
- The Date object can be used date and time in terms of millisecond precision within 100 million dates before or after 1/1/1970. But sing another methods we can only get and set the year, month, date, hour, minutes second and millisecond fields using local or UTC of GMT time.
- So, we can declare date in different ways, the basic things is that the date objects are created by the
  - new Date()
  - new Date(year, month, day, hours, minutes, seconds, milliseconds)
  - new Date(milliseconds)
  - new Date(date string)

Date object

- **new Date():** Date() constructor creates a Date object which sets the current date and time depend on the browser's time zone. It does not accepts any value.
- **new Date(milliseconds):** this method accepts single parameter milliseconds which indicates any numeric value. This argument is taken as the internal numer**New Date(datastring)**: this ,method accepts single parameter data string which indicates any string value. It is a string representation of a date and return the datastring with the day.
- **New Date(year, month, day, hours, minutes, seconds, milliseconds)**
- This method accepts seven parameters as mentioned above and described below.
- Year: integer value which represents the year. Use full year, 2018 instead of 18 only.
- Month: integer value which represents the month. The integer value are starts from 0 for January to 11 for December.
- Date: integer value which represents the date

Date object

- Hour: integer value which represents the hour in 24 hours scale.
- Minute: integer value which represents the minute
- Second: integer value which represents the second
- Millisecond: integer value which represents the millisecond.

Date object methods

Date Get Methods

Method	Description
getFullYear()	Get **year** as a four digit number (yyyy)
getMonth()	Get **month** as a number (0-11)
getDate()	Get **day** as a number (1-31)
getDay()	Get **weekday** as a number (0-6)
getHours()	Get **hour** (0-23)
getMinutes()	Get **minute** (0-59)
getSeconds()	Get **second** (0-59)
getMilliseconds()	Get **millisecond** (0-999)
getTime()	Get **time** (milliseconds since January 1, 1970)

RegExp object

- In JavaScript, a **Reg**ular **Ex**pression (RegEx) is an object that describes a sequence of characters used for defining a search pattern. It provide more precise search pattern.
- **Create a RegExp object:**
- There are two ways you can create a regular expression in JavaScript.

1. **Using a regular expression literal:**

   The regular expression consists of a pattern enclosed between slashes **/**. For example,

   ```
   const regularExp = /abc/;
   ```

   Here, /abc/ is a regular expression.

2. **Using the RegExp() constructor function:**

   You can also create a regular expression by calling the RegExp() constructor function. For example,

   ```
   const regularExp = new RegExp('abc');
   ```

   For example,

   ```
   const regex = new RegExp(/^a...s$/);

   console.log(regex.test('alias')); // true
   ```

   In the above example, the string alias matches with the RegEx pattern /^a...s$/. Here, the test() method is used to check if the string matches the pattern.

RegEx object methods

- Syntax : /pattern/modifier(s);
- Let b=/nccscollege/i;

- Regexr.com to try and learn all variations of regex

Modifiers

Modifiers define how to perform the search:

Modifier	Description
/g	Perform a global match (find all)
/i	Perform case-insensitive matching
/m	Perform multiline matching

RegEx object methods

Eg. Text="My favorite number is 7";

document.write(text.match([0-9]); // selects the numbers between 0 to 9 i.e. returns 7

Brackets

Brackets are used to find a range of characters:

Bracket	Description
[abc]	Find any character between the brackets
[^abc]	Find any character NOT between the brackets
[0-9]	Find any character between the brackets (any digit)
[^0-9]	Find any character NOT between the brackets (any non-digit)
(x|y)	Find any of the alternatives specified

RegEx object methods

Metacharacters

Metacharacters are characters with a special meaning:

Character	Description
.	Find a single character, except newline or line terminator
\w	Find a word character
\W	Find a non-word character
\d	Find a digit
\D	Find a non-digit character
\s	Find a whitespace character
\S	Find a non-whitespace character
\b	Find a match at the beginning/end of a word, beginning like this: \bHI, end like this: HI\b
\B	Find a match, but not at the beginning/end of a word
\0	Find a NULL character
\n	Find a new line character
\f	Find a form feed character
\r	Find a carriage return character
\t	Find a tab character
\v	Find a vertical tab character
\xxx	Find the character specified by an octal number xxx
\xdd	Find the character specified by a hexadecimal number dd
\udddd	Find the Unicode character specified by a hexadecimal number dddd

RegEx object methods

Quantifiers

Quantifier	Description
n+	Matches any string that contains at least one *n*
n*	Matches any string that contains zero or more occurrences of *n*
n?	Matches any string that contains zero or one occurrences of *n*
n{X}	Matches any string that contains a sequence of *X* *n*'s
n{X,Y}	Matches any string that contains a sequence of X to Y *n*'s
n{X,}	Matches any string that contains a sequence of at least X *n*'s
n$	Matches any string with *n* at the end of it
^n	Matches any string with *n* at the beginning of it
?=n	Matches any string that is followed by a specific string *n*
?!n	Matches any string that is not followed by a specific string *n*

Window object

- The window object is supported by all browsers. It represents the browser's window. All global JavaScript objects, functions and variables automatically become members of the window object. Two properties can be used to determine the size of browsers window.
- Both properties return sizes in pixels.
- window.innerHeight: specifies inner height of browsers window
- window.innerwidth: specifies inner width of the browser's window

Methods	Description
window.open()	Open a new window
window.close()	Close the current window
window.moveTo()	Move to current window
window.resizeTo()	Resize the current window

Window object

```
<script>
    let Window;

    // Function that open the new Window
    function windowOpen() {
        Window = window.open(
            "https://www.geeksforgeeks.org/",
            "_blank", "width=400, height=300, top=230, left=540");
    }

    // function that Closes the open Window
    function windowClose() {
        Window.close();
    }
</script>
```

Event Handling

- We can use events in javascript same as CSS. Check CSS slide for that.
- JavaScript addEventListener()
- The addEventListener() method is used to attach an event handler to a particular element. It does not override the existing event handlers. An event listener is a JavaScript's procedure that waits for the occurrence of an event.
- The addEventListener() method is an inbuilt function of JavaScript. We can add multiple event handlers to a particular element without overwriting the existing event handlers.
- Syntax
- element.addEventListener(event, function);
- **event:** It is a required parameter. It can be defined as a string that specifies the event's name.
- **function:** It is also a required parameter. It is a JavaScript function which responds to the event occur.

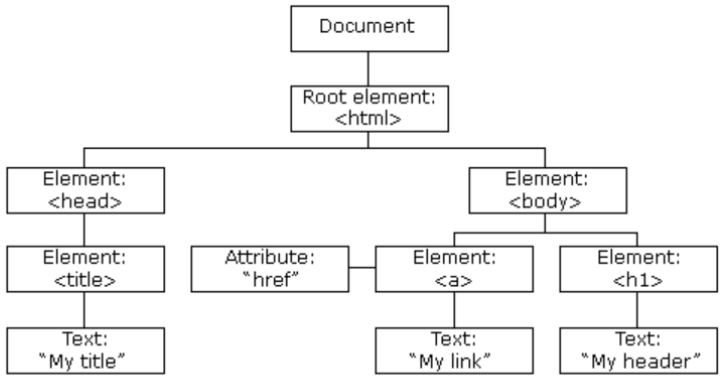
Event handling Eg

```
<html>
<head>
    <meta charset="UTF-8" />
    <title>title</title>
</head>
<body>
 <button id="bid">Check Time</button>
    <script>
        function showtime()
        {
            var d=new Date();
            var hr=d.getHours();
            var min=d.getMinutes();
            var sec=d.getSeconds();
            var time=hr+':'+min+':'+sec;
            document.getElementById('pid1').innerHTML=time;

        }


            function welcome()
            {
                document.getElementById('pid').innerHTML="Welcome to Event Listener";
            }
            var element1=document.getElementById('bid');
            element1.addEventListener("click",showtime);
            element1.addEventListener("click",welcome);
            </script>
            <p id="pid">Welcome message here</p><br>
        <p id="pid1">Time Displayed here</p>
</body>
</html>
```

The HTML DOM (Document Object Model)

- With the HTML DOM, JavaScript can access and change all the elements of an HTML document.
- When a web page is loaded, the browser creates a **D**ocument **O**bject **M**odel of the page.
- The **HTML DOM** model is constructed as a tree of **Objects**:

The HTML DOM Tree of Objects



Here are some key concepts related to the DOM in JavaScript:

- Document: The document object represents the HTML document loaded in the browser. It serves as the entry point to access and manipulate the DOM tree.

- Element: Elements represent the individual HTML tags within the document. They are nodes in the DOM tree and can be accessed using methods like getElementById, getElementsByTagName, or querySelector.

- Node: Nodes are the building blocks of the DOM tree. They can represent elements, text, comments, or other types of nodes. Nodes have various properties and methods to traverse the DOM tree and interact with other nodes.

- Event: The DOM provides a way to handle events triggered by user actions or other sources, such as clicks, key presses, mouse movements, and more. You can attach event handlers to elements to respond to these events using methods like addEventListener.

Form validation

- Forms in web pages are sent to server for processing. If the data in form are incorrect then it may put lot of burden in server. So JavaScript provides a way to validate forms data on the clients computer before sending it to the web server.
- Form validation generally performs two functions:
  - Basic validation: First of all, the form must be checked to make sure all the mandatory fields are filled in.
  - Data format validation: Secondly, the data that is entered must be checked for correct form and value.

Error Handling in JavaScript

What is an Error?

An error is an event that occurs during the execution of a program that prevents it from continuing normally. Errors can be caused by a variety of factors, such as syntax errors, runtime errors, and logical errors.

Syntax Errors

Logical Errors

Runtime Error

Error Handling in JavaScript

What is Error Handling?

Whenever any error occurs in the JavaScript code, the JavaScript engine stops the execution of the whole code. If you handle such errors in the proper way, you can skip the code with errors and continue to execute the other JavaScript code.

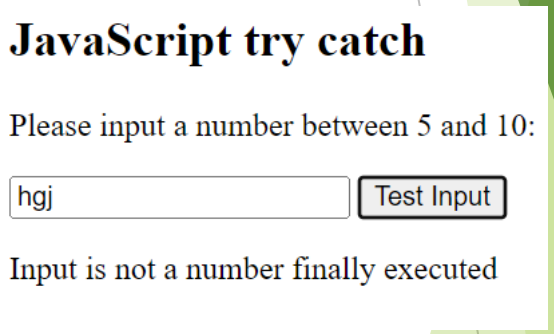
Error Handling in JavaScript

Throw, and Try...Catch...Finally

- The try statement defines a code block to run (to try).
- The catch statement defines a code block to handle any error.
- The finally statement defines a code block to run regardless of the result.
- The throw statement defines a custom error.

Example

```
<h2>JavaScript try catch</h2>
<p>Please input a number between 5 and 10:</p>
<input id="demo" type="text">
<button type="button" onclick="myFunction()">Test Input</button>
<p id="p01"></p>
<script>
function myFunction() {
  const message = document.getElementById("p01");
  message.innerHTML = "";
  let x = document.getElementById("demo").value;
  try {
    if(x.trim() == "")  throw "empty";
    if(isNaN(x)) throw "not a number";
    if(x < 5)  throw "too low";
    if(x > 10)   throw "too high";
  }
  catch(err) {
    message.innerHTML = "Input is " + err;
  }
  finally{
   message.innerHTML += " finally executed";
  }
}
</script>
```



JavaScript Cookies

- Cookies are data, stored in small text files, on your computer.
- Cookies let you store user information in web pages.
- When a web server has sent a web page to a browser, the connection is shut down, and the server forgets everything about the user.
- Cookies were invented to solve the problem "how to remember information about the user":
- When a user visits a web page, his/her name can be stored in a cookie.
- Next time the user visits the page, the cookie "remembers" his/her name.

JavaScript Cookies

- Cookies are saved in name-value pairs like:
- username = John Doe
- When a browser requests a web page from a server, cookies belonging to the page are added to the request. This way the server gets the necessary data to "remember" information about users.

- None of the examples below will work if your browser has local cookies support turned off.

How It Works ?

- Your server sends some data to the visitor's browser in the form of a cookie. The browser may accept the cookie. If it does, it is stored as a plain text record on the visitor's hard drive. Now, when the visitor arrives at another page on your site, the browser sends the same cookie to the server for retrieval. Once retrieved, your server knows/remembers what was stored earlier.

Create a Cookie with JavaScript

- JavaScript can create, read, and delete cookies with the document.cookie property.

- With JavaScript, a cookie can be created like this:
- **document.cookie = "username=John Doe";**

- You can also add an expiry date (in UTC time). By default, the cookie is deleted when the browser is closed:
- **document.cookie = "username=John Doe; expires=Thu, 18 Dec 2013 12:00:00 UTC";**

- With a path parameter, you can tell the browser what path the cookie belongs to. By default, the cookie belongs to the current page.
- **document.cookie = "username=John Doe; expires=Thu, 18 Dec 2013 12:00:00 UTC; path=/";**

Read a Cookie with JavaScript

- With JavaScript, cookies can be read like this:
- let x = document.cookie;
- document.cookie will return all cookies in one string much like: cookie1=value; cookie2=value; cookie3=value;

Change a Cookie with JavaScript

- With JavaScript, you can change a cookie the same way as you create it:

- document.cookie = "username=John Smith; expires=Thu, 18 Dec 2013 12:00:00 UTC; path=/";

- The old cookie is overwritten.

Delete a Cookie with JavaScript

- Deleting a cookie is very simple
- You don't have to specify a cookie value when you delete a cookie.
- Just set the expires parameter to a past date:
- document.cookie = "username=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;";

JQuery

- jQuery is a JavaScript library that makes web pages quicker, easier, responsive and handler the request.
- Often with jQuery you can write a single line of code to achieve what would have taken 10-20 lines of regular JavaScript code.
- jQuery allows web developers to plug routine JavaScript features into a web page so they can spend more time focusing on complicated features that are unique to their site.
- **Things you can do with jQuery:**
  1. Adding animated effects to elements like fading in/out, sliding in/out and expanding/contracting.
  2. Making XML requests.
  3. Manipulating DOM. You can easily add, remove and reorder content in the web pages using just a couple of lines of code.
  4. Creating image slideshows
  5. Making Drop-down menus
  6. Creating drag and drop interfaces
  7. Adding complex client side form validation.

JQuery Syntax

- **$(selector).methodorFunction();**

- All statements come from a basic template for applying functions and methods to selected HTML elements and their attributes. This method can be easily modified depending on the selector and the action which is the applied method or function.
- **Document Ready Event**
- Before jQuery is applied, it is important to make sure that all HTML elements have been loaded and the CSS is fully computed. These conditions are necessary for jQuery to be applied properly. To make sure that is the case, we place all jQuery inside the document ready event. This means that the statements inside the block of code will only be applied once the web page has finished loading.
- **$(document).ready(()=>{**

```
        $('.fast').hide();              //All jQuery statements go here
});
```

jQuery Selectors

- Selectors in jQuery are meant to specify a set of elements based on certain attributes such as ID, class, or the type of tag itself. These elements can then be selected of applying the jQuery method or a function you define.
- **Selecting by Type:(element name)**
  - $(“a”) // a tag
- **Selecting by Class:**
  - $(“.example”) // example being class name of HTML element
- **Selecting by ID:**
  - $(“#example1”) //example1 being ID name of HTML element

jQuery Selectors

- **Selecting by Possession of Attribute**
  - $(“[href]”) // selects the HTML element(a tag) with href as attribute
- **Selecting by Attribute Value:**
  - $(“[href=‘index.html’]”)
  - //selects the HTML element with href attribute and index.HTML as value
- **Selecting by ID:**
  - $(“#example1”) //example1 being ID name of HTML element
- **Selecting by Pseudo state**
  - You can also select in jQuery using pseudo states including :first-child, :last-child, :first-of-type, :last-of-type, etc
  - $(“a:first-of-type”) //will select first a tag element among series of a tags.

jQuery Selectors

- **Combining jQuery selectors**
  - $(“a.class1.class2#someid[attribute][attribute=‘something’”) OR
  - $(“a,.class1,#someID”)
- **Wildcard selection(Universal):**
  - There might be cases when we want to select all elements but there is not a common property to select upon(class, attribute, etc). In that case we can use the * selector that simply selects all the elements.
  - $(‘#wrapper *’) //selects all elements inside #wrapper element

jQuery Events

Mouse Events	Keyboard Events	Form Events	Document/Window Events
click	keypress	submit	load
dblclick	keydown	change	resize
mouseenter	keyup	focus	scroll
mouseleave		blur	unload

jQuery Effects

i) **hide/show:** with jQuery we can hide and show HTML elements with the hide() and show() methods:

Syntax: $(selector).hide(speed, callback);

Syntax: $(selector).hide(speed, callback);

- The optional speed parameter specifies the speed of the hiding or showing and can take values : “slow”, “fast” or milliseconds. The optional callback parameter is a function to be executed after the hide() or show() method completes.

Example: $(“button”).click(function(){

```
$(“p”).hide(1000);      //hide content under p tag with 1000 millisecond speed on clicking button
});
```

jQuery Effects

ii. fade:

With jQuery we can fade an element in and out of visibility. jQuery has fadeIn(), fadeOut(), fadeToggle(), fadeTo() fade methods. Let us take jQuery fadeIn() method.

Syntax: $(selector).fadeIn(speed, callback);

Example: $("button").click(function(){

```
$("p").fadeIn(1000);  //fades content under p tag with 1000 millisecond speed on clicking button
});
```

jQuery Effects

iii. Slide:

The jQuery slide methods slide elements up and down. It creates sliding effect on elements. jQuery has slideDown(), slideUp(), slideToggle() methods. Let us take slideDown() method.

Syntax: $(selector).slideDown(speed, callback);

Example: $("button").click(function(){

$("p").slideDown(1000); //slides content under p tag with 1000 millisecond speed on clicking button

});

jQuery Effects

iv. Animate:

We can create custom animations using jQuery.

Syntax: $(selector).animate({parameter},speed, callback);

The parameters specifies the CSS properties to be animated.

Example: $(“button”).click(function(){

$(“p”).animate({left: ‘250px’});

});

jQuery Effects

v. stop():

stop() method is used to stop animations or effects before it is finished. This method words for all jQuery effect functions, including sliding, fading and custom animations.

Syntax: $(selector).stop({parameter}, stopAll, goToEnd);

The optional stopAll parameter specifies whether also the animation queues should be cleared or not, default is false. The optional goToEnd parameter specifies whether or not to complete the current animation immediately, default is false.

Example: $(“button”).click(function(){

$(“#div”).stop();

});

jQuery Effects

vi. callback():

stop() function is executed after the current effect is finished.

Syntax: $(selector).hide(speed,callback);

Example: $("button").click(function(){

```
        $("p").hide("slow",function(){
                alert("The paragraph is now hidden");
        });
});
```

jQuery Effects

vii. chaining():

Chaining allows us to run multiple jQuery commands one after another on the same element(s). To chain an action we simply append the action to the previous action. The following example chains together the css(), slideUp() and slideDown() methods.

Example: $(“button”).click(function(){

$(“#div”).css(“color”, “red”).slideUp(2000),slideDown(2000);

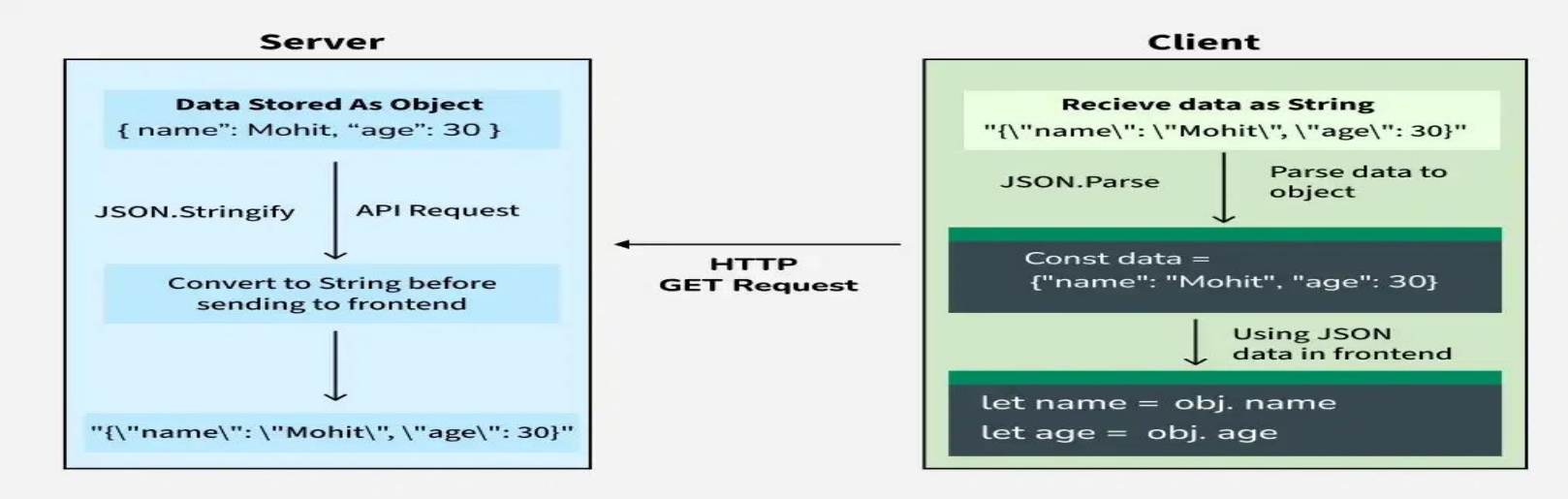
});

Introduction to JSON

JSON (JavaScript Object Notation) is a text-based, human-readable data interchange format used to exchange data between web clients and web servers. The format defines a set of structuring rules for the representation of structured data. JSON is used as an alternative to Extensible Markup Language (XML).

JSON is used in JavaScript on the internet as an alternative to XML for organizing data. JSON is language-independent and can be combined with C++, Java, Python and many other languages. Unlike XML, which is a full markup language, JSON is simply a way to represent data structures. JSON documents are relatively lightweight and are rapidly executed on web servers.

JSON Data Flow: From Server to Client



In a typical web application, the server sends data to the client (frontend) as JSON.

Server Side:

- Data is stored as a JavaScript object.
- Before sending the data to the client, it's converted to a JSON string using JSON.stringify().

Client Side:

- The JSON string is received as part of an API response (e.g., via an HTTP GET request).
- The client parses this string back into a JavaScript object using JSON.parse().
- The parsed object is then used in the frontend code.

Convert Object into JSON

Transferring Objects from server side to client side

Convert Object into JSON

```
let obj = {name: "Mohit", age: 30};
let jsonS= JSON.stringify(obj);
console.log(jsonS);
```

Convert JSON into object

Here’s a JSON string received from the server we need to parse it into a JavaScript object:

- const jsonS = '{"name":"Mohit", "age":30}’;
- const obj1 = JSON.parse(jsonS);
- let name = obj1.name;
- let age = obj1.age;
- console.log(`Name: ${name}, Age: ${age}`);

- **Key Features of JSON**
  - **Text-based:** JSON is a simple text format, making it lightweight and easy to transmit.
  - **Human-readable:** It uses key-value pairs, making the structure easy to understand.
  - **Language-independent:** While it is derived from JavaScript, JSON is supported by many programming languages including Python, Java, PHP, and more.
  - **Data structure:** It represents data as objects, arrays, strings, numbers, booleans, and null.
- **JSON Structure**
- The basic structure of JSON consists of two primary components:
  - **Objects:** These are enclosed in curly braces {} and contain key-value pairs.
  - **Arrays:** Arrays are ordered lists enclosed in square brackets [].
- **Applications of JSON**
  - **APIs:** JSON is the most commonly used format for API responses due to its lightweight nature.
  - **Configuration Files:** Many software systems use JSON files for storing configuration data.
  - **Databases:** Some NoSQL databases, like MongoDB, store data in JSON-like formats.
  - **Data Transfer:** JSON is widely used for transferring data between servers and clients, especially in web development.